

图像风格迁移与分类

刘东琛 赖星宇 张皓哲

2025 年 3 月 25 日

1 概述

图像的风格作为图像的一种重要特征，在计算机视觉领域是一个重要的课题。我们围绕图像的风格主题，对神经风格迁移和图像风格分类问题进行了一些探索和实现。

整个项目的源代码在 github 上开源：[我们的 github 仓库](#)

2 神经风格迁移

神经风格迁移旨在将风格图像的纹理，色彩等特征迁移整合到内容图像的结构信息中，从而生成艺术化内容图像的目的。本次实践我们复现探索了传统的基于图像迭代的迁移算法 (Gatys et al.) 和基于前馈生成网络的快速神经迁移方法 (Johnson et al.)，探索了二者的优缺点。

2.1 基本原理

在卷积神经网络被用于图像的分类和识别时，实际上这个网络的各个卷积层中得到的结果就包含着图像中的各种内容信息。具体来说经过更多次的卷积池化操作后的图像显现出来的信息就更加抽象，所以浅层的网络中就会包含更多的内容和结构信息，而更深层的网络中就会包含抽象的纹理和色彩信息。于是我们可以提取各卷积层的内容信息，并使用较为浅层的图像信息定义内容损失，使用较为深层的信息定义风格损失，从而获得一个可以通过梯度下降求解最小值的损失函数。

2.1.1 内容损失函数

定义内容损失函数的朴素想法就是直接对图像的每个像素求均方误差。但逐像素进行会陷入对原图的重复，实际操作中使用高层卷积层中的特征图的均方误差。即：

$$L_{content} = \frac{1}{2} \sum_{i,j} (G_{ij} - C_{ij})^2$$

其中的 G 代表生成图像 (generate_img), C 代表内容图像 (content_img) 同时论文中给出了损失函数的求导公式：

$$\frac{\partial L_{content}}{\partial G_{ij}} = \begin{cases} (G - C)_{ij} & \text{if } G_{ij} > 0 \\ 0 & \text{if } G_{ij} < 0 \end{cases}$$

实现过程中通过 torch 库中的函数自动求导得到。

2.1.2 风格损失函数

在 Gatys 的论文中认为，卷积层的特征图对应的 gram 矩阵能反映通道之间的相似度，从而很好地表现出图像的风格特征。对于一个 RGB 格式的图像，我们将其表示为 $img(chanel, height, width)$ ，将这个图像的按照通道展开可以得到一个矩阵

$$F_{ij} = [chanel][height * width]$$

定义

$$Gram_{ij} = FF^T = \sum_k F_{ik} F_{jk}$$

为这个图像对应的 gram 矩阵。接下来就可以定义风格损失函数，我们令 S 为风格图像的 gram 矩阵，X 为输入图像的风格 gram 矩阵，则有：

$$E_l = \frac{1}{4ch_l^2 * H_l^2 W_l^2} \sum_{i,j} (X_{ij}^l - A_{ij}^l)^2$$

$$L_{style} = \sum_l w_l E_l$$

其中 l 代表在不同卷积层的采样 w_l 代表各层的采样权值, $\frac{1}{4ch_l^2 * H_l^2 W_l^2}$ 是归一化因子分别表示图像的通道数、高和宽。论文中也给出了这一部分的求导公

式：

$$\frac{\partial L_{style}}{\partial X_{ij}} = \begin{cases} \frac{1}{ch_l^2 * H_l^2 * W_l^2} ((F^l)^T X^l - A^l)_{ji} & \text{if } F_{ij}^l > 0 \\ 0 & \text{if } F_{ij}^l < 0 \end{cases}$$

2.1.3 梯度下降法迭代

有了上面的讨论，现在我们可以定义总损失函数：

$$L_{total} = \alpha L_{content} + L_{style}$$

其中的 α 为内容损失值和风格损失值的比值。

2.2 基于图像迭代优化的传统算法

传统算法的优化对象是输入的内容图像，输入图像经过卷积网络后在指定层计算内容损失和风格损失，得到的总损失函数就是输入图像的函数，通过对损失函数求最小值就可以得到整合内容和风格后的图像。

2.2.1 数据预处理

为方便损失函数的计算，我们需要统一输入图像、内容图像和风格图像的大小。统一大小之后再将其转为 pytorch 中便于处理的 tensor 张量，并在给图片添加一个 batch 维度，以统一后续模型处理。并且使用 pytorch 张量常用的参数进行了标准化。

这一部分通过 cv 库和 torch 库函数实现。

2.2.2 函数设计

复现传统方法实现时，参考了[pytorch 官方神经风格迁移教程](#)。在定义内容损失和风格损失函数时并没有以函数的形式计算给出，而是继承自 torch.nn.Module 并重构了 forward 函数，实际运行时将损失计算层添加到模型中的特定位置，实现了前向传播中自动计算出损失函数的值。

2.2.3 迭代求解

传统方法的优化对象是输入的图像，需将用于提取特征的模型切换为评测模式，使用 LBFGS 优化器对输入的图像进行了优化。

有关于输入图像的选取，Gatys 在论文中使用了随机生成的白噪声图像来初始化优化图像，实际操作中发现使用内容图像的复制来初始化图像会更快得到收敛的结果。

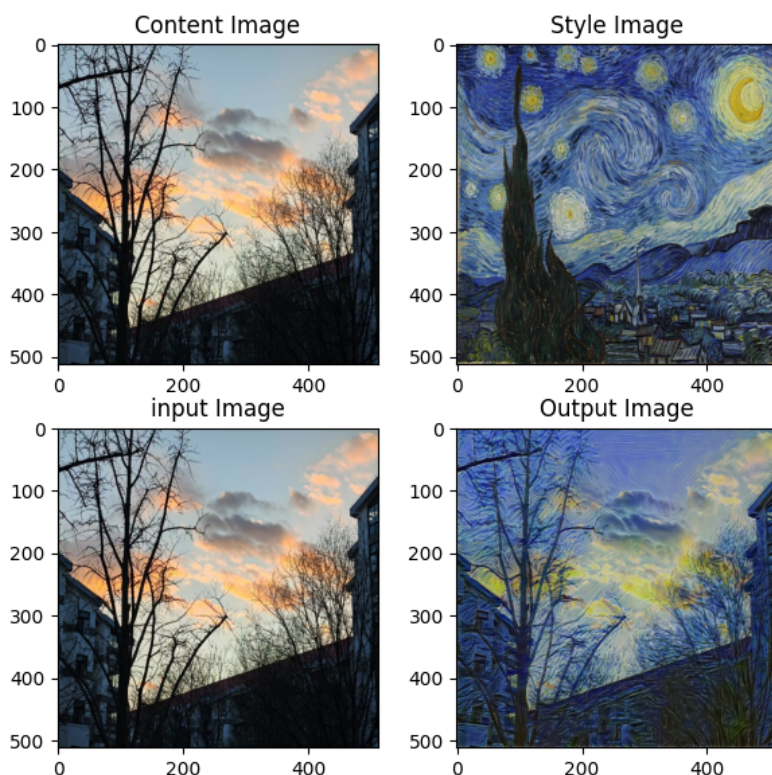


图 1: 传统方法的示例效果

2.2.4 传统方法的困境

正如上面所提到的，传统的求解方法的过程实际上是针对输入的图像的参数进行的优化，这意味着每产生一次输出都需要进行多次的迭代，这个过程相对耗时，对一张 515×512 的图片进行 300 次迭代在我的个人电脑上就需要 30s。并且由于需要对原图像进行迭代计算，且原图像分辨率越高时这个输出时间会迅速增大，这几乎是不能接受的。回顾这个过程我们可以想

到，如果将优化对象从输入的图像改变为一个生成网络，从而就可以保存这个模型的参数，避免了每次生成图像都需要迭代计算的问题。

2.3 使用生成网络的快速迁移方案

针对上面提到的困境，Johnson et al. 提出了一种使用生成网络的快速迁移方法。这个方法在输入图像和计算损失函数之间添加了一个生成网络。

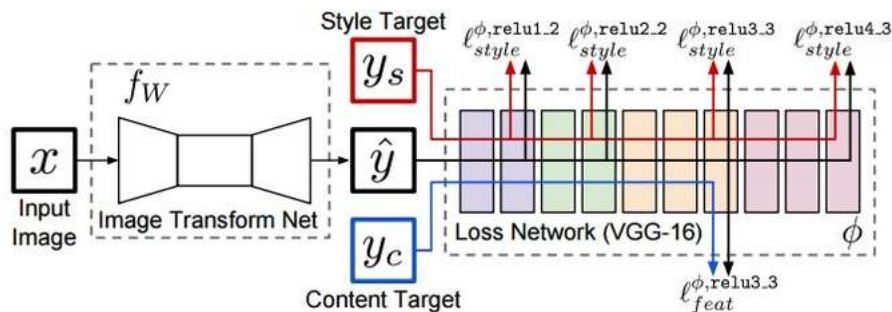


图 2: 快速神经迁移网络的基础结构

这样原来的损失函数可以表示为生成网络 f_{gene} 的函数

$$L_{total}(f_{gene}(x)) = L_{total}(\hat{y})$$

在迭代过程中就可以针对 f_{gene} 进行梯度优化得到一个可以生成有效图片的生成网络模型。

2.3.1 生成网络的构建

生成网络本质是还是一个卷积神经网络，但其目标是生成一个图像而不是提取特征和分类，所以这个模型没有池化层，实际上是一个深度残差网络。其主要结构为：

下卷积采样模块 $\rightarrow$ 残差块 $\rightarrow$ 上卷积恢复模块

1. 下卷积采样模块：

由于我们希望图像的大小在卷积过程中保持不变，我们需要在卷积前对各层图像进行边缘填充。torch.nn.Conv2d 的默认填充方式是 0 填充，但是对于风格迁移任务，图像中的边缘纹理特征可能相对重要，所以在网络中实际采用了反射填充方式，使其对边缘敏感的特征保留更加有效。

2. 残差块:

残差块的前向传播函数如下:

```
1 def forward(self, x):  
2     res = x  
3     out = self.block(x)  
4     out = out + res  
5     return out
```

这样的设计通过将原始输入叠加到输出上,保证了内容信息不会再深层网络中被迅速破坏,同时允许了网络专注学习风格残差,并且通过叠加缓解了传统深层网络中的梯度消失退化问题,提升了生成图像的质量

3. 上卷积恢复模块:

上卷积模块的任务是将深度网络中的特征层恢复为一幅图像,恢复过程需要对像素进行填充和恢复,具体实现时通过反射填充和最近采样恢复来实现。

2.3.2 对生成网络训练

1. 数据集导入:

根据生成网络的结构可知,我们只需要针对特定风格,提供大量的内容图像以供学习即可。在实现中使用了 COCO-2014 数据集进行训练。由于训练中并不需要标注信息,复现时我们设计了一个 FolderDataset 导入类,并实现了在导入数据时对数据进行统一大小、转为张量、标准化操作。

2. 训练生成网络:

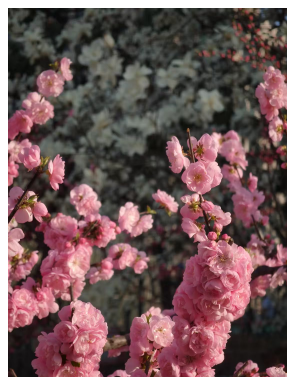
定义损失函数的方法与前面的原理相似,其中 $L_{content}$ 通过 VGG16 卷积层中的 4-9 层计算, L_{style} 对四部分的 relu 函数都进行计算。这里采用 Adam 函数对损失函数进行优化。由于在训练过程中观察到 loss 降低的速度很快,并且很早就出现震荡,于是设置了指数学习率衰减机制,每处理 1k 张图片衰减为原来的 0.9 倍。

2.3.3 使用训练得到的生成网络实现风格化

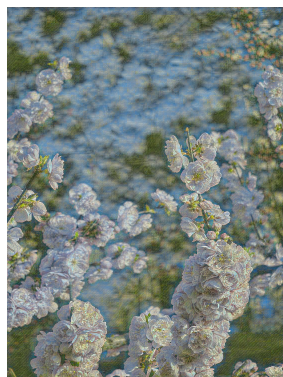
训练后得到一系列风格的模型参数,使用时只需导入模型参数,让输入的图像经过生成网络即可得到风格化后的图像。对于分辨率在 1000*1000 的图像,在我的个人电脑上仅需要 0.5s 即可得到结果。对于一张 3072*4096 的大分辨率图像,也仅需 20s 左右就可以完成风格化,所需的时间大大下降了。

2.3.4 一些结果的展示

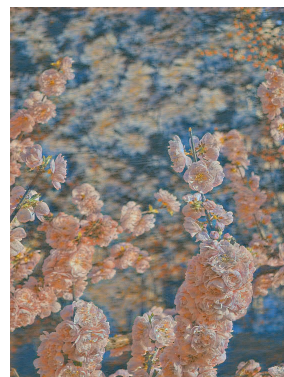
使用个人相册里面的一些图像作为内容图像，得到了一些比较好的结果；



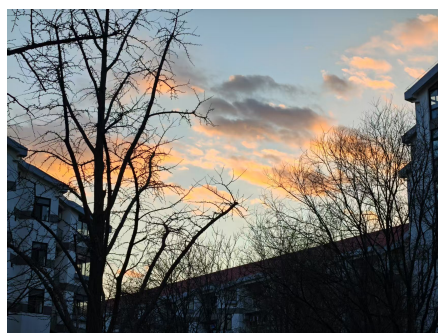
(a) 原图像



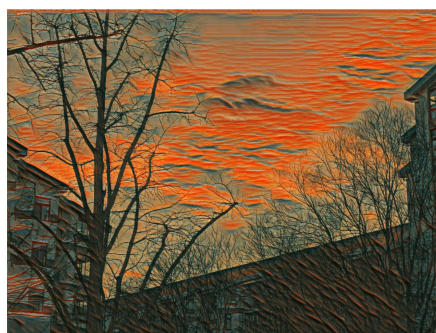
(b) 莫奈风



(c) 日出印象风



(a) 原图像



(b) scream 风



(a) 原图像



(b) wave 风

我们制作了 GUI 界面可以供读者体验，读者可在我们的 github 仓库中体验一些已经预训练的风格生成网络。

2.4 遇到的问题与改进策略

1. 生成图像效果难以评估

生成的图像风格化的效果难以通过给定的参数进行评估，只能通过观察得到，对于基于生成神经网络的方法而言，每次观察模型效果需要耗费大量的训练时间。这为复现过程中参数的调整和模型结果的微调造成了很大的困扰。因此，我参考了开源项目[快速风格迁移](#)中对于生成网络和 VGG16 中提取层数的设计。

原计划使用下一部分中的图像分类模型来进行特征提取和结果评估，但由于图像分类中实际分类目标与风格迁移中的风格 = 风格相差较大，这个尝试最终失败了。

2. 不同风格图像所需的参数结果差异较大：

对于不同的风格图像，其对于风格和内容的权重极其敏感，同样的参数可能在不同的图像上效果差异较大。

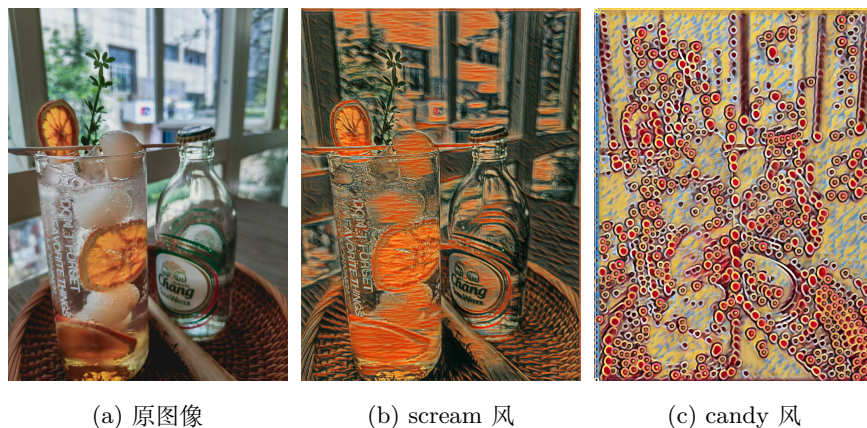


图 6: 同样权重参数下不同风格的结果差异

由于计算资源和时间有限等问题，没有进行更加细节的调参工作。

3. 快速风格迁移效果相对较差

基于图像迭代的实现方法对于内容图像的针对性较强，而在快速风格迁移中需要对大量的内容图像找到兼容的参数，并且收到数据集大小和算力的限制，不能针对更大规模的图像进行训练，这使得目前我实现的快速神经风格迁移效果不如传统方法。在 Johnson et al. 的论文中得到的效果则比传统方法好。造成这一结果的原因可能是前面提到的参数问题和训练所用的数据导致。



图 7: 不同方法输出的结果差异

4. 可能的改进方向:

笔者对风格迁移任务进行了进一步的调研后了解到，基础的神经风格

迁移模型提出之后，已经有了很多的发展。有基于 GAN（生成式对抗神经网络）和 diffusion 模型的现代神经迁移风格方案，可以实现更好的效果和更少的计算时间。

3 图像风格分类

本实验采用自创的 DualPathResNet18-UNet 模型在 wikiart 数据集上实现画作类型的分类任务。通过数据增强、学习率调整等优化手段，最终在测试集达到 67.71% 的准确率，验证该模型在该分类任务中的有效性。

3.1 背景

本研究选取了目前能找到的最大的艺术作品数据集 wikiart, 但是 wikiart 往往被用于其他领域。目前网上只能找到一个正确率仅为 49% 的将其用于图像分类的工作 (<https://github.com/mbellitti/wikiart-classifier>)，为了解决相应的图像分类问题，急需提出一个新的网络架构来提高分类的准确率。

3.2 模型架构

一开始我们考虑使用原始的 resnet18(He et al.)（图 8，图 9）来进行训练：

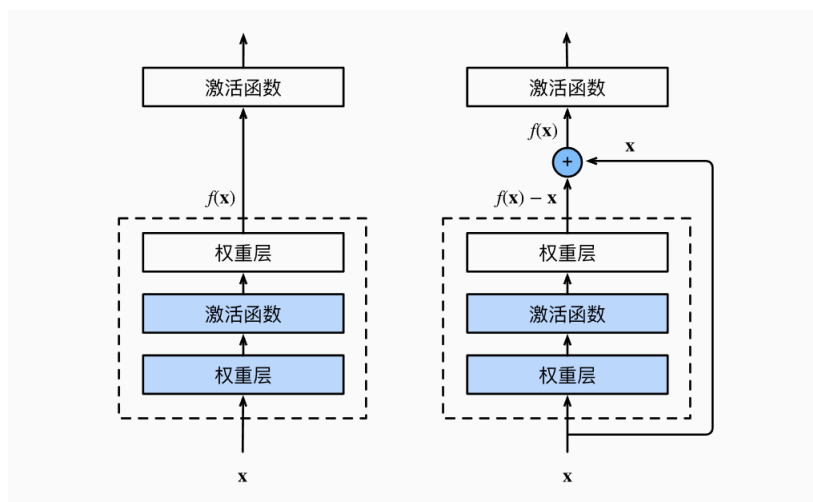


图 8: 一个正常块（左）和一个残差块（右）

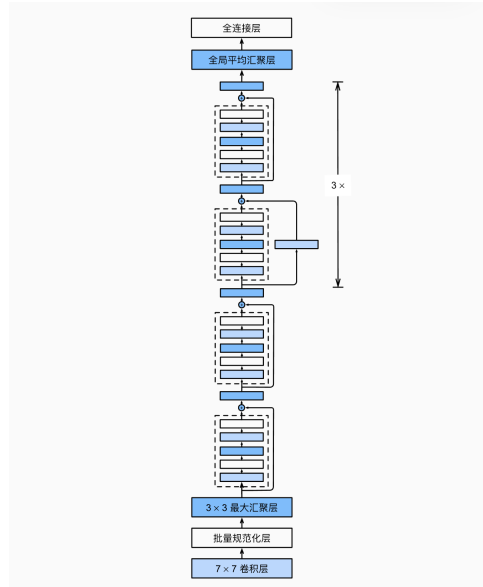


图 9: resnet18 架构

但是考虑到在 wikiart 数据集中，有许多黑白画作并没有颜色的特征，故可以结合 UNet(Ronneberger et al.) 可以用于提取图像边沿的特点，一定程度上提高 resnet18 的性能。改造后的模型将一个浅层的 UNet 按照图 10 中的方式与 resnet18 相连。

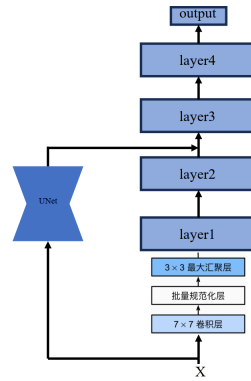


图 10: DualPathResNet18-UNet

本来我们在连接 UNet 和 Resnet18 的时候采用了 SE 注意力 (Hu et al.) (图 11)，但是发现加入了 SE 注意力的模型比没有加入 SE 注意力的模型在测试集上的准确率大约下降了 2%。推测可能是数据集不够大，导致 SE-block 难以学习到一个合适的参数来提高性能或者模型在训练集上过拟合程度加深了。

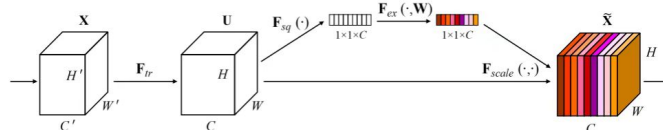


图 11: SE 注意力

3.3 实验过程

3.3.1 数据预处理

为了使得模型更快收敛并且提升模型的泛化能力，在训练集、验证集和测试集上我们分别采取了以下方式对数据进行预处理。

```

1 #这是对训练集
2 transform = transforms.Compose([
3     transforms.Resize((256, 256)),
4     transforms.ToTensor(),
5     transforms.RandomHorizontalFlip(),
6     transforms.RandomRotation(15),
7     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
8     0.224, 0.225]),
9 ])

```

```

1 #这是对验证集以及测试集
2 transform = transforms.Compose([
3     transforms.Resize((256, 256)),
4     transforms.ToTensor(),
5     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
6     0.224, 0.225]),
7 ])

```

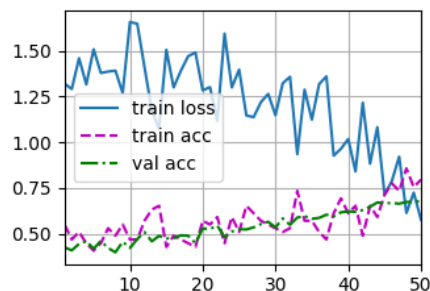


图 12: 我们的模型训练时的损失、准确率的变化

3.3.2 超参数配置

在还没有确定使用 wikiart 数据集进行图像分类的时候，笔者在 cifar10 数据集上先进行训练达到了 94% 并将之前的经验迁移到该次实验，最终采用了以下超参数。

```

1 learning_rate = 3e-2
2 epochs = 50
3 weight_decay = 3e-3
4 # 优化器
5 optimizer_net = optim.SGD(net.parameters(), lr=learning_rate, momentum=0.9,
6                             weight_decay=weight_decay)
7 # 采用余弦退火，减少陷入局部最优的可能性
8 scheduler=optim.lr_scheduler.CosineAnnealingLR(optimizer_net, T_max=50)

```

3.3.3 训练时长

该模型训练 50epochs 大概需要在一台 3090 的服务器运行 7 个小时。

3.4 实验结果

训练时的损失、以及准确率的变化见图 5，最终在测试集上达到了 67.71% 的准确率。

3.5 消融实验

训练原始的 resnet18 时的损失以及准确率的变化见图 6。该模型最终在测试集上达到了 66.81% 的准确率。

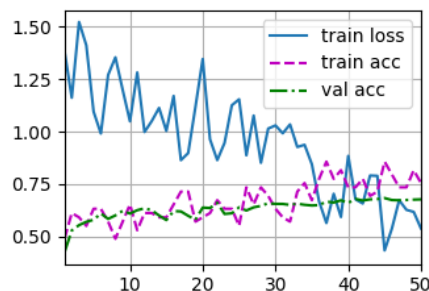


图 13: resnet18 训练时的损失、准确率的变化

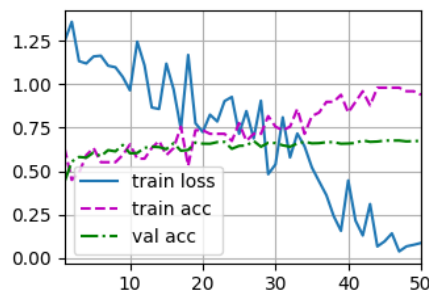


图 14: 我们的模型与训练 resnet18 时 weight-decay 相近时训练的损失、准确率的变化

我们的模型在测试集上准确率只提高了 1% 左右。两者准确率差别不大，一方面很可能是因为 resnet18 模型在训练的时候可以自动学习到提取图像边沿的方式，另一方面也可能是训练集与测试集的偏差过大的问题，当我们的 DualPathResNet18UNet 的 weight-decay 调到与训练 resnet18 时相近的时候（图 7），在测试集上大概能达到 66.92% 的准确率，而在训练集上可以达到接近 100% 的准确率，但是增大 weight-decay 减少过拟合并不能显著提高测试集的准确率。如果测试集与训练集分布更加相近，极大可能提高该模型的性能。

3.6 改进方向

一方面，可能可以采用更加深层或高效的网络来提高准确率。另一方面，可以搜集更加适合分类的艺术作品数据集（wikiart 数据集主要被用于

风格迁移), 来提高模型的性能。

4 Dataset

图片分类和风格迁移任务通常需要大量高质量的数据用于训练。我们选择了以画作风格为主题的“Wikiart”网站作为数据来源。然而, 该网站缺少标准化的数据集结构和可直接用于训练的 Dataloader, 因此我们对其数据进行了预处理, 以满足模型训练的需求。

4.1 数据集处理

我们从 Wikiart 网站下载了近 8 万张带有风格标签的图片, 并编写脚本将其按 1:8:1 的比例随机划分为训练集 (Trainset)、测试集 (Testset) 和验证集 (Valset)。同时, 我们将每张图片的风格信息存入 JSON 文件, 以便于训练, 并构建了适用于 PyTorch 的 WikiArtDataset 类。随后, 我们编写了适用于 PyTorch 的 Dataloader, 确保每次能够高效地提供 batchsize 张 RGB 图片及其对应的标签, 以支持模型训练。特别的, 我们对图片均进行归一化处理, 将像素值映射到 $[0, 1]$ 区间, 有助于梯度下降过程中的学习效率。

重新制作的数据集结构如下:

```
wikiart/  
  train/  
    images/  
      00010-of-00072/image0.jpg.....  
      00011-of-00072/image0.jpg.....  
      ...  
    labels/  
      00010-of-00072.json  
      00011-of-00072.json  
      ...  
  test/  
    images/  
      00000-of-00072/image0.jpg.....  
      00001-of-00072/image0.jpg.....
```

```

...
labels/
    00000-of-00072.json
    00001-of-00072.json
    ...
val/
    images/
        00005-of-00072/image0.jpg.....
        00006-of-00072/image0.jpg.....
        ...
    labels/
        00005-of-00072.json
        00006-of-00072.json
        ...

```

以上部分均通过 torch 中的 Dataset 和 Dataloader 实现。

5 GUI 界面

我们利用 tkinter 库制作了较为简单的可视化界面, 支持用户自选任意的风格后, 上传本地的图片进行风格迁移和分类, 并可视化生成的图片

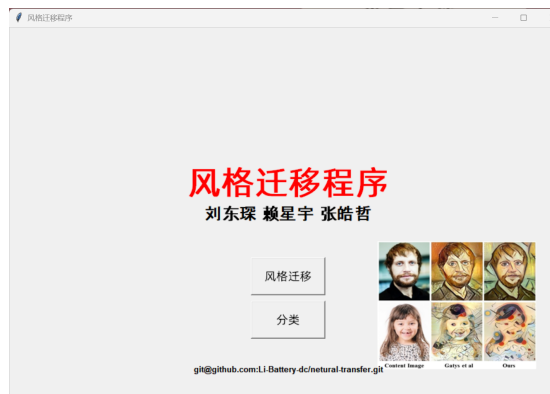


图 15: 主菜单

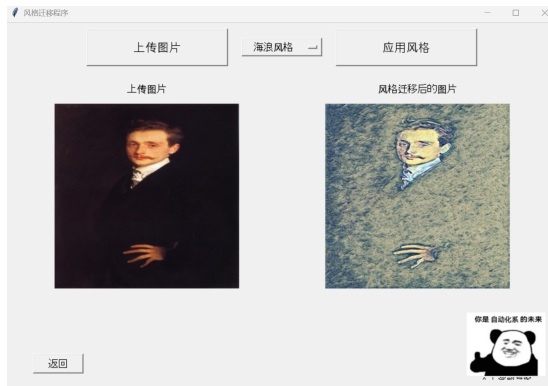


图 16: 风格迁移界面 1

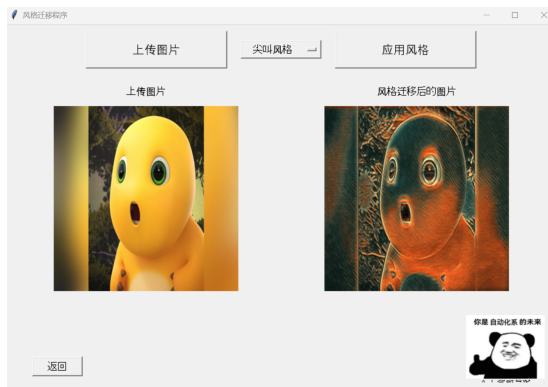


图 17: 风格迁移界面 2



图 18: 分类界面

6 一些说明

6.1 参考文献

Gatys, L. A., Ecker, A. S., & Bethge, M. (2015). A neural algorithm of artistic style. arXiv preprint arXiv:1508.06576.

Johnson, J., Alahi, A., & Fei-Fei, L. (2016). Perceptual losses for real-time style transfer and super-resolution. In Computer Vision –ECCV 2016 (pp. 694-711). Springer, Cham. https://doi.org/10.1007/978-3-319-46475-6_43

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 770-778). IEEE. <https://doi.org/10.1109/CVPR.2016.90>

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. In Medical Image Computing and Computer-Assisted Intervention (MICCAI) (pp. 234-241). Springer, Cham. https://doi.org/10.1007/978-3-319-24574-4_28

6.2 代码来源说明

1. 神经风格迁移部分：

风格迁移模型的代码部分参考了 github 中的一些开源项目以及知乎文章中的一些讲解。并且一些基础的图像预处理和数据集导入部分在 ai 生成代码的基础上修改得到。

2. 图像分类部分：

图像分类模型的代码是笔者提出想法，并让 DeepSeek 实现的。笔者只是把 DeepSeek 生成的代码里面的 bug 改动了。

3. 数据集处理部分：数据集的重构、Dataloader、GUI 等均为笔者在 GPT 的协助下完成。

6.3 图像来源说明

展示生成网络的图 2 来自 Johnson et al. 的原论文。

展示网络结构的图 8、图 9 以及图 10 中的部分,来自 <https://zh.d2l.ai/>网站。图 11 来自 <https://blog.csdn.net/renxingshen2022/article/details/125773673> 网站。

6.4 分工说明

刘东琛完成了报告中的神经风格迁移部分,赖星宇完成了报告中的图像风格分类部分,张皓哲完成了报告中的 Dataset 和 GUI 界面部分。